

NHIỆM VỤ OJT – CLEAN CODE & REFACTOR (01 TUẦN)

Mục tiêu

Fresher sẽ nhận **source code Order API** (Java 17, Spring Boot 3.x, JPA, PostgreSQL) và **tự refactor toàn bộ hệ thống** dựa trên **5 tiêu chí kỹ thuật cốt lõi**:

1. SOLID Principles
2. Design Patterns
3. DRY (Don't Repeat Yourself)
4. KISS (Keep It Simple, Stupid)
5. Clean Code

Source code ban đầu **cố tình chứa nhiều vấn đề** (code smells).

Nhiệm vụ của Fresher là **nhận diện – phân tích – refactor** một cách có hệ thống.

Kế hoạch làm việc chi tiết (5 ngày – làm 1 mình)

DAY 1 – ĐỌC CODE & PHÂN TÍCH VẤN ĐỀ

Mục tiêu

- Hiểu business flow của Order API
- Nhận diện các vấn đề thiết kế & clean code

Công việc cần làm

1. Clone repo và chạy thành công:
docker compose up -d
mvn spring-boot:run
mvn test
2. Đọc kỹ các file sau (bắt buộc):
 - OrderServiceImpl
 - OrderService
 - OrderController
 - DTO & Entity
 - Discount logic trong createOrder
3. Tạo file: docs/refactor-notes.md
4. Liệt kê **ít nhất 10 vấn đề**, phân loại theo:
 - SOLID
 - Design Patterns
 - DRY
 - KISS
 - Clean Code

Ví dụ mong đợi (không giới hạn):

- Method `createOrder()` quá dài
- Dùng magic string ("NEW", "vip", "PROMO10")
- Service vừa xử lý business vừa gửi email
- Discount dùng if-else kéo dài
- Interface `OrderService` quá “béo”

Output ngày 1

- docs/refactor-notes.md (bản phân tích, chưa code)

DAY 2 – SOLID: ISP & SRP**Mục tiêu**

Áp dụng **Single Responsibility Principle** và **Interface Segregation Principle**

Công việc cần làm

1. Refactor interface `OrderService`
 - Tách thành các interface nhỏ, ví dụ:
 - `OrderCommandService` (create order)
 - `OrderQueryService` (get order)
 - `Notifier`
 - `OrderExporter`
2. Refactor `OrderServiceImpl`
 - Không còn “God Service”
 - Mỗi class **chỉ làm 1 việc chính**
3. Controller **chỉ inject interface cần dùng**, không inject service “đa năng”

Yêu cầu kỹ thuật

- Không thay đổi API behavior
- Code compile & test pass

Output ngày 2

- Commit rõ ràng:
refactor: apply SRP & ISP to order services

DAY 3 – DESIGN PATTERN: STRATEGY (DISCOUNT)

Mục tiêu

Loại bỏ if–else phức tạp, áp dụng **Strategy Pattern**

Công việc cần làm

1. Thiết kế interface:
interface DiscountStrategy {
 int calculate(int totalCents, CreateOrderRequest request);
}
2. Tạo các strategy riêng biệt:
 - Promo10DiscountStrategy
 - Promo20DiscountStrategy
 - FreeShipDiscountStrategy
 - VipCustomerDiscountStrategy
 - NewCustomerDiscountStrategy
3. Tạo DiscountStrategyRegistry hoặc DiscountService
 - Map strategy theo discountCode / customerType
 - Không dùng if–else trong service chính
4. OrderService chỉ gọi: discountService.calculateDiscount(...)

Output ngày 3

- Discount logic mở rộng được mà **không sửa code cũ**
- Commit message rõ ràng

DAY 4 – DESIGN PATTERN: OBSERVER (EVENT-DRIVEN)

Mục tiêu

Loại bỏ side-effect trong business logic

Công việc cần làm

1. Tạo domain event: OrderCreatedEvent(orderId, customerEmail, total, discount)
2. Khi tạo order thành công:
 - Publish event bằng ApplicationEventPublisher
4. Tạo listener:
 - OrderEmailNotificationListener
 - Gửi email thông qua Notifier

5. OrderService:

- **Không gửi email trực tiếp**
- Chỉ tập trung business logic

Output ngày 4

- OrderService sạch, không side-effect
- Áp dụng Observer Pattern đúng chuẩn Spring

DAY 5 – DRY + KISS + CLEAN CODE

Mục tiêu

Hoàn thiện code theo chuẩn production

Công việc cần làm

DRY

- Loại bỏ code trùng lặp
- Gom validation hợp lý
- Tạo enum cho:
 - OrderStatus
 - CustomerType
 - DiscountCode

KISS

- Tách nhỏ method
- Tên method rõ nghĩa
- Logic dễ đọc, dễ trace

Clean Code

- Không dùng System.out.println
- Logging có context (orderId, email)
- DTO dùng:
 - record (Java 17) **hoặc**
 - private fields + getter/setter
- Thêm JavaDoc ngắn cho public API quan trọng

Testing (bắt buộc)

- Thêm **2–3 test case**:
 - Discount strategy
 - Order creation happy path

Output ngày 5

- mvn test pass
- Code clean, dễ đọc
- PR sẵn sàng review

Deliverables cuối tuần

Fresher phải nộp:

1.  Pull Request refactor
2.  docs/refactor-notes.md (đã cập nhật đầy đủ)
3.  Test pass
4.  API chạy được, không đổi behavior

Tiêu chí đánh giá của Mentor

Tiêu chí	Mức đánh giá
Hiểu đúng vấn đề	★★★★
Áp dụng SOLID	★★★★
Design Pattern đúng chỗ	★★★
Code dễ đọc	★★★★
Commit rõ ràng	★★★
Tư duy refactor	★★★★